Mastering JUnit & Spring Boot Testing -- Architecture, Patterns, and Enterprise Usage Guide

---

Author: Senior Java Test Architect **Target Audience:** Java backend developers with 3+ years experience aiming for architect-level testing expertise **Prerequisites:** Java, Spring Boot fundamentals, basic SQL knowledge

---

Table of Contents

1. Software Testing Fundamentals
2. JUnit Architecture
3. JUnit Test Lifecycle
4. JUnit Assertions
5. JUnit Parameterized Tests
6. Mockito Mocking Framework
7. Spring Boot Testing Architecture
8. Testing Controller Layer
9. Testing Service Layer
10. Testing Repository Layer
11. Testcontainers
12. Spring Security Testing
13. Code Coverage
14. CI/CD Testing Integration
15. Production Testing Best Practices
16. Common Testing Mistakes
17. Complete Project Structure
18. Interview Questions (100+)
19. Hands-on Practice Exercises

Part 1: Software Testing Fundamentals

1.1 What is Software Testing?

Concept Explanation

Software testing is the process of evaluating a software application to detect differences between expected and actual behavior. It is NOT just "checking if the code works" -- it is a systematic verification that the software meets business requirements, handles edge cases, and behaves correctly under failure scenarios.

Why Testing Exists

In enterprise systems -- banking, insurance, healthcare, e-commerce -- a single bug can cause:

- **Financial loss** -- incorrect calculations in payment processing
- **Data corruption** -- wrong database updates affecting thousands of records
- **Security breaches** -- unvalidated input leading to SQL injection
- **Regulatory violations** -- non-compliant behavior in government systems

Testing exists to **catch these problems before they reach production**.

The Cost of Finding Bugs

```
Phase Where Bug is Found          Cost to Fix
---------------------------------------------
During Development (Unit Test)    $1
During Integration Testing        $10
During QA / Staging               $100
In Production                     $1,000 - $10,000+
```

> ***Key Insight:*** *The later a bug is found, the exponentially more expensive it is to fix. Unit tests are the cheapest and fastest way to find bugs.*

1.2 The Testing Pyramid

Concept Explanation

The **Testing Pyramid** is a strategy that defines how many tests of each type you should write. More tests at the bottom (fast, cheap), fewer at the top (slow, expensive).

```
                  /\
                 /  \
                / E2E \        Few tests (Slow, Expensive)
               /  Tests \      Full system, browser, network
              /----------\
             /            \
            / Integration  \   Medium tests
           /    Tests       \  Multiple components together
          /------------------\
         /                    \
        /     Unit Tests       \  Many tests (Fast, Cheap)
       /    (70-80% of tests)   \ Single class/method in isolation
      /--------------------------\
```

Why This Pyramid Shape?

Level	Speed	Cost	Reliability	Quantity
Unit Tests	Milliseconds	Very Low	Very High	70-80%
Integration Tests	Seconds	Medium	High	15-20%
E2E Tests	Minutes	High	Medium (flaky)	5-10%

Real-World Use Case

In a banking application:

- **Unit Test:** Verify that the `calculateInterest()` method returns correct values for different rates
- **Integration Test:** Verify that the `TransferService` correctly debits one account and credits another through the repository layer
- **E2E Test:** Verify that a user can log in, navigate to transfer page, enter details, and complete a transfer

1.3 Types of Testing

Unit Testing

What: Testing a single class or method in **complete isolation**. All dependencies are mocked.

Why: Fastest feedback loop. You know exactly which method broke and why.

```
+----------------------------------------+
|           Unit Test Scope              |
|                                        |
|   +--------------+                     |
|   |  OrderService |  <- Class Under Test |
|   |  .placeOrder()|                     |
|   +------+-------+                     |
|          |                             |
|   +------▼-------+  +--------------+   |
|   |  Mock         |  |  Mock         |   |
|   |  Repository   |  |  PaymentClient|   |
|   +--------------+  +--------------+   |
|                                        |
|   Real class tested, dependencies mocked|
+----------------------------------------+
```

Integration Testing

What: Testing multiple components working together. Uses real database, real Spring context.

Why: Verifies that components integrate correctly -- SQL queries work, transactions commit, beans wire properly.

System Testing

What: Testing the entire application as a deployed unit. Includes API testing, database validation, message queue verification.

End-to-End Testing

What: Testing the complete user journey from UI to database and back.

Why: Catches issues that only appear when all systems work together.

The Role of Unit Testing in Backend Systems

```
Why Unit Tests are the Foundation:

1. SPEED      -> Run 1000 tests in seconds (not minutes)
2. ISOLATION  -> When a test fails, you know EXACTLY which method broke
3. DESIGN     -> Writing testable code forces better architecture
```

```
4. CONFIDENCE -> Refactor fearlessly knowing tests will catch regressions
5. DOCUMENTATION -> Tests describe what the code SHOULD do
```

> ***Architect's Rule:*** *If your service layer method cannot be unit tested easily, your design has a problem. Testability is a sign of good architecture.*

Part 2: JUnit Architecture

2.1 What is JUnit?

Concept Explanation

JUnit is the standard testing framework for Java. It provides annotations, assertions, and test runners that enable developers to write and execute automated tests.

Why JUnit Exists

Before JUnit, developers tested by:

- Adding `main()` methods with `System.out.println()` -- not automated, not repeatable
- Manual testing -- slow, error-prone, not scalable

JUnit provides a **standardized, automated, repeatable** way to verify code behavior.

2.2 Evolution: JUnit 4 to JUnit 5

Why JUnit 5 Was Created

JUnit 4 had limitations:

- Monolithic architecture (single JAR for everything)
- Required `@RunWith` for extensions (only one runner allowed)
- No support for lambda expressions
- Limited parameterized test support

Feature	JUnit 4	JUnit 5
Architecture	Monolithic	Modular (Platform + Jupiter + Vintage)
Extensions	`@RunWith` (single)	`@ExtendWith` (multiple)
Assertions	Basic	Rich (lambdas, grouped, timeout)
Parameterized	Limited	Multiple sources (@ValueSource, @CsvSource, @MethodSource)
Display names	Class/method name only	`@DisplayName` for readable names

Nested tests	Not supported	`@Nested` for test grouping
Minimum Java	Java 5	Java 8+

2.3 JUnit 5 Platform Architecture

Architecture Diagram

```
+-------------------------------------------------------+
|                    IDE / Build Tool                     |
|              (IntelliJ, Eclipse, Maven, Gradle)         |
+----------------------+--------------------------------+
                       | Discovers and launches tests
                       ▼
+-------------------------------------------------------+
|                  JUnit Platform                         |
|          (Foundation for launching testing              |
|           frameworks on the JVM)                        |
|                                                         |
|  +-------------+  +-------------+  +--------------+    |
|  | Launcher API|  | TestEngine  |  | Extension    |    |
|  |             |  | SPI         |  | Model        |    |
|  +-------------+  +------+------+  +--------------+    |
+--------------------------+-----------------------------+
                           | Implementations
             +------------+------------+
             ▼            ▼            ▼
   +--------------+ +-----------+ +--------------+
   | JUnit Jupiter| |JUnit      | | Third-party  |
   | (JUnit 5     | |Vintage    | | Engines      |
   |  tests)      | |(JUnit 3/4 | | (TestNG,     |
   |              | | tests)    | |  Spock, etc.) |
   +--------------+ +-----------+ +--------------+
```

Component Responsibilities

Component	Role	When You Use It
JUnit Platform	Foundation layer -- discovers and executes tests	Always (transparent -- Maven/Gradle handle it)

JUnit Jupiter	The JUnit 5 API -- annotations, assertions, extensions	When writing NEW tests
JUnit Vintage	Backward compatibility engine for JUnit 3/4 tests	When migrating LEGACY tests

Real-World Use Case

When you add this Maven dependency:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

Spring Boot automatically includes JUnit Jupiter + Mockito + AssertJ + Hamcrest + JSONassert. You get the full testing stack with a single dependency.

Part 3: JUnit Test Lifecycle

3.1 Concept Explanation

What is the Test Lifecycle?

The **test lifecycle** defines the order in which setup methods, test methods, and teardown methods are executed. Understanding this order is critical for managing shared resources (database connections, test data, external services).

Why the Lifecycle Exists

Tests need:

- **One-time setup** -- start embedded database, load configuration (expensive, do once)
- **Per-test setup** -- reset test data, clear state (ensure test isolation)
- **Per-test teardown** -- clean up resources after each test
- **One-time teardown** -- shut down embedded database, release connections

3.2 Lifecycle Annotations

Annotation	When It Runs	How Many Times	Use Case
`@BeforeAll`	Once before ALL tests in the class	1 time	Start database, load config
`@BeforeEach`	Before EACH test method	N times (once per test)	Reset test data, clear mocks
`@Test`	The actual test	1 time per method	Verify behavior
`@AfterEach`	After EACH test method	N times	Clean up test data
`@AfterAll`	Once after ALL tests in the class	1 time	Shut down database, close connections

3.3 Execution Flow Diagram

```
Class: OrderServiceTest (3 test methods)
```

```
+-----------------------------------------------+
| @BeforeAll -- static setup()                  |  Runs ONCE
|   -> Start embedded DB, load test config      |
+-----------------+-------------------------+
                  |
+-----------------▼-------------------------+
| @BeforeEach -- setUp()                        |  Runs before test1
|   -> Insert test data, reset mocks            |
+-----------------------------------------------+
| @Test -- test_placeOrder_success()            |  Test 1
+-----------------------------------------------+
| @AfterEach -- tearDown()                      |  Runs after test1
|   -> Delete test data                         |
+-----------------+-------------------------+
                  |
+-----------------▼-------------------------+
| @BeforeEach -- setUp()                        |  Runs before test2
+-----------------------------------------------+
| @Test -- test_placeOrder_insufficientStock()  |  Test 2
+-----------------------------------------------+
| @AfterEach -- tearDown()                      |  Runs after test2
+-----------------+-------------------------+
                  |
+-----------------▼-------------------------+
| @BeforeEach -- setUp()                        |  Runs before test3
+-----------------------------------------------+
| @Test -- test_placeOrder_invalidCustomer()    |  Test 3
+-----------------------------------------------+
| @AfterEach -- tearDown()                      |  Runs after test3
+-----------------+-------------------------+
                  |
+-----------------▼-------------------------+
| @AfterAll -- static cleanup()                 |  Runs ONCE
|   -> Shut down embedded DB                    |
+-----------------------------------------------+
```

3.4 Code Implementation

```java
import org.junit.jupiter.api.*;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class OrderServiceLifecycleTest {

    private static DatabaseConnection dbConnection;
```

```java
    private OrderService orderService;
    private OrderRepository orderRepository;

    @BeforeAll
    static void globalSetup() {
        // Runs ONCE before all tests
        // Use for expensive one-time operations
        dbConnection = new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2)
            .addScript("schema.sql")
            .build();
        System.out.println("Database started");
    }

    @BeforeEach
    void setUp() {
        // Runs before EACH test -- ensures clean state
        orderRepository = new OrderRepository(dbConnection);
        orderService = new OrderService(orderRepository);

        // Insert fresh test data
        orderRepository.save(new Order(1L, "PENDING",
BigDecimal.valueOf(100)));
        System.out.println("Test data inserted");
    }

    @Test
    @DisplayName("Should place order successfully")
    void testPlaceOrderSuccess() {
        Order result = orderService.placeOrder(new OrderRequest(1L, 2));
        assertNotNull(result);
        assertEquals("CONFIRMED", result.getStatus());
    }

    @Test
    @DisplayName("Should throw exception for invalid order")
    void testPlaceOrderInvalidInput() {
        assertThrows(IllegalArgumentException.class,
            () -> orderService.placeOrder(null));
    }

    @AfterEach
    void tearDown() {
        // Runs after EACH test -- clean up
        orderRepository.deleteAll();
        System.out.println("Test data cleaned up");
    }
```

```
    @AfterAll
    static void globalTeardown() {
        // Runs ONCE after all tests
        dbConnection.close();
        System.out.println("Database stopped");
    }
}
```

Code Explanation

Line	What It Does	Why
`@BeforeAll static`	Must be static -- runs before any instance is created	Class-level setup
`@BeforeEach setUp()`	Creates fresh service/repository instances	Ensures no state leaks between tests
`@Test`	Marks the actual test method	JUnit discovers and runs these
`@DisplayName`	Human-readable test name	Better reports, easier debugging
`@AfterEach`	Cleans up after each test	Test isolation
`@AfterAll static`	Must be static -- runs after all instances	Class-level cleanup

Common Mistakes

Mistake	Problem	Fix
`@BeforeAll` not static	Compilation error	Make the method `static`
Shared mutable state between tests	Tests pass individually, fail together	Reset all state in `@BeforeEach`
Heavy setup in `@BeforeEach`	Tests run slowly	Move expensive setup to `@BeforeAll`

Part 4: JUnit Assertions

4.1 Concept Explanation

What are Assertions?

Assertions are the verification statements in your tests. They compare the **actual result** of your code against the **expected result**. If they don't match, the test **fails**.

Why Assertions Exist

Without assertions, a test would just execute code without verifying anything. Assertions are the **"then"** in the Arrange-Act-Assert pattern:

```
1. ARRANGE  ->  Set up test data and dependencies
2. ACT      ->  Call the method under test
3. ASSERT   ->  Verify the result matches expectations
```

4.2 Core Assertions Reference

Assertion	Purpose	When to Use
`assertEquals(expected, actual)`	Values are equal	Verifying return values, calculations
`assertNotEquals(a, b)`	Values are NOT equal	Verifying values changed
`assertTrue(condition)`	Condition is true	Boolean checks, validations
`assertFalse(condition)`	Condition is false	Negative validations
`assertNull(value)`	Value is null	Verifying cleanup, optional returns
`assertNotNull(value)`	Value is NOT null	Verifying object creation
`assertThrows(Exception.class, executable)`	Exception is thrown	Error handling verification
`assertDoesNotThrow(executable)`	No exception thrown	Happy path verification

`assertAll(executables...)`	Multiple assertions grouped	Verifying multiple fields at once
`assertTimeout(duration, executable)`	Completes within time	Performance verification

4.3 Code Examples with Scenarios

```
class UserServiceAssertionTest {

    private UserService userService;

    @BeforeEach
    void setUp() {
        userService = new UserService(new InMemoryUserRepository());
    }

    // assertEquals: Verify return values
    @Test
    @DisplayName("Should return user by ID")
    void testFindUserById() {
        User user = userService.findById(1L);

        assertEquals("John Doe", user.getName());
        assertEquals("john@company.com", user.getEmail());
        assertEquals(UserStatus.ACTIVE, user.getStatus());
    }

    // assertThrows: Verify exception handling
    @Test
    @DisplayName("Should throw exception for non-existent user")
    void testFindUserNotFound() {
        UserNotFoundException exception = assertThrows(
            UserNotFoundException.class,
            () -> userService.findById(999L)
        );
        assertEquals("User not found with id: 999", exception.getMessage());
    }

    // assertAll: Group related assertions
    @Test
    @DisplayName("Should create user with all fields populated")
    void testCreateUser() {
        CreateUserRequest request = new CreateUserRequest(
            "Jane", "jane@company.com", "PREMIUM");
```

```java
        User created = userService.create(request);

        // All assertions run even if one fails
        assertAll("created user",
            () -> assertNotNull(created.getId()),
            () -> assertEquals("Jane", created.getName()),
            () -> assertEquals("jane@company.com", created.getEmail()),
            () -> assertEquals("PREMIUM", created.getPlan()),
            () -> assertEquals(UserStatus.ACTIVE, created.getStatus()),
            () -> assertNotNull(created.getCreatedAt())
        );
    }

    // assertTrue/assertFalse: Boolean conditions
    @Test
    @DisplayName("Should validate email format")
    void testEmailValidation() {
        assertTrue(userService.isValidEmail("test@example.com"));
        assertFalse(userService.isValidEmail("invalid-email"));
        assertFalse(userService.isValidEmail(null));
    }

    // assertTimeout: Performance verification
    @Test
    @DisplayName("Should find user within 100ms")
    void testPerformance() {
        assertTimeout(Duration.ofMillis(100), () -> {
            userService.findById(1L);
        });
    }
}
```

Best Practices

1. **Always provide a message for complex assertions** -- `assertEquals(expected, actual, "Order total should include tax")`
2. **Use `assertAll` for DTO/entity verification** -- see all failures at once instead of fixing one at a time
3. **Use `assertThrows` instead of try-catch** -- cleaner, captures the exception for further assertions
4. **Assert behavior, not implementation** -- test what the method returns, not how it internally works

---

Part 5: JUnit Parameterized Tests

5.1 Concept Explanation

What are Parameterized Tests?

Parameterized tests allow you to run the **same test logic** with **different input data**. Instead of writing 10 separate test methods with similar logic, you write ONE method and feed it different parameters.

Why Parameterized Tests Exist

Without parameterization:

```
// BAD: 5 methods testing the same logic with different inputs
@Test void testIsValidEmail_validEmail() {
assertTrue(validator.isValid("a@b.com")); }
@Test void testIsValidEmail_noAtSign() {
assertFalse(validator.isValid("invalid")); }
@Test void testIsValidEmail_noDomain() { assertFalse(validator.isValid("a@"));
}
@Test void testIsValidEmail_empty() { assertFalse(validator.isValid("")); }
@Test void testIsValidEmail_null() { assertFalse(validator.isValid(null)); }
```

With parameterization:

```
// GOOD: 1 method, 5 data sets
@ParameterizedTest
@CsvSource({"a@b.com, true", "invalid, false", "a@, false", "'', false"})
void testIsValidEmail(String email, boolean expected) {
    assertEquals(expected, validator.isValid(email));
}
```

When to Use Parameterized Tests

- **Boundary value testing** -- testing edge cases (0, -1, MAX_VALUE)
- **Equivalence partitioning** -- testing one value from each valid/invalid category
- **Data-driven tests** -- same logic, many data combinations
- **Validation rules** -- testing input validation with valid and invalid inputs

5.2 Annotations Reference

Annotation	Data Source	Use Case
`@ValueSource`	Inline values (strings, ints, etc.)	Simple single-parameter tests
`@NullSource`	null value	Testing null handling
`@EmptySource`	Empty string/collection	Testing empty input
`@NullAndEmptySource`	Both null and empty	Combined null/empty testing
`@CsvSource`	Inline CSV rows	Multi-parameter tests
`@CsvFileSource`	External CSV file	Large test datasets
`@MethodSource`	Factory method	Complex objects, dynamic data
`@EnumSource`	Enum values	Testing all enum cases

5.3 Code Examples

```
class OrderValidationParameterizedTest {

    private OrderValidator validator = new OrderValidator();

    // @ValueSource: Single parameter, multiple values
    @ParameterizedTest
    @DisplayName("Should reject negative order amounts")
    @ValueSource(doubles = {-1.0, -0.01, -100.0, -999999.99})
    void testRejectNegativeAmounts(double amount) {
        assertFalse(validator.isValidAmount(BigDecimal.valueOf(amount)));
    }

    // @CsvSource: Multiple parameters per test case
    @ParameterizedTest
    @DisplayName("Should calculate discount correctly")
    @CsvSource({
        "100.00, BASIC,      0.00",    // No discount for BASIC
        "100.00, PREMIUM,    10.00",   // 10% for PREMIUM
        "100.00, ENTERPRISE, 20.00",   // 20% for ENTERPRISE
        "0.00,   PREMIUM,    0.00",    // Zero amount = zero discount
        "999.99, ENTERPRISE, 200.00"   // Large amount discount
    })
    void testCalculateDiscount(BigDecimal amount, String plan,
```

```java
                                BigDecimal expectedDiscount) {
        BigDecimal discount = validator.calculateDiscount(amount, plan);
        assertEquals(expectedDiscount, discount);
    }

    // @MethodSource: Complex objects
    @ParameterizedTest
    @DisplayName("Should validate order request")
    @MethodSource("provideInvalidOrders")
    void testInvalidOrders(OrderRequest request, String expectedError) {
        ValidationResult result = validator.validate(request);
        assertFalse(result.isValid());
        assertEquals(expectedError, result.getErrorMessage());
    }

    static Stream<Arguments> provideInvalidOrders() {
        return Stream.of(
            Arguments.of(
                new OrderRequest(null, 1, BigDecimal.TEN),
                "Customer ID is required"),
            Arguments.of(
                new OrderRequest(1L, 0, BigDecimal.TEN),
                "Quantity must be positive"),
            Arguments.of(
                new OrderRequest(1L, 1, BigDecimal.ZERO),
                "Amount must be positive"),
            Arguments.of(
                new OrderRequest(1L, 1001, BigDecimal.TEN),
                "Quantity exceeds maximum limit")
        );
    }

    // @EnumSource: Test all enum values
    @ParameterizedTest
    @DisplayName("All order statuses should have a display name")
    @EnumSource(OrderStatus.class)
    void testAllStatusesHaveDisplayName(OrderStatus status) {
        assertNotNull(status.getDisplayName());
        assertFalse(status.getDisplayName().isEmpty());
    }
}
```

Best Practices

1. **Use `@CsvSource` for simple multi-parameter tests** -- readable and inline
2. **Use `@MethodSource` for complex objects** -- when CSV can't represent your data

3. **Name your test clearly** -- use `@DisplayName` to explain what's being tested
4. **Include edge cases** -- null, empty, zero, negative, max values

---

Part 6: Mockito Mocking Framework

6.1 What is Mocking?

Concept Explanation

Mocking is the practice of creating **fake implementations** of dependencies so you can test a class in **complete isolation**. Instead of calling the real database, HTTP client, or external service, you create a mock that returns predefined responses.

Why Mocking is Needed

```
Problem WITHOUT Mocking:

  OrderService.placeOrder()
        |
        +-- calls -> ProductRepository.findById()    -> Needs REAL database
        +-- calls -> PaymentClient.charge()           -> Needs REAL payment
gateway
        +-- calls -> EmailService.sendConfirmation()  -> Sends REAL emails

  To test OrderService, you need:
  ✗ A running database with test data
  ✗ A payment gateway sandbox
  ✗ An email server
  Result: Slow, fragile, expensive tests

Problem WITH Mocking:

  OrderService.placeOrder()
        |
        +-- calls -> MOCK ProductRepository           -> Returns fake Product
instantly
        +-- calls -> MOCK PaymentClient                -> Returns fake
PaymentResponse
        +-- calls -> MOCK EmailService                 -> Does nothing (verified
later)

  Result: Fast, reliable, isolated tests
```

6.2 Core Mockito Concepts

Concept	What It Does	When to Use
Mock	Fake object that returns default values (null, 0, false)	Replace dependencies you don't want to call
Stub	Mock configured to return specific values	When the method under test needs specific input
Spy	Wraps a REAL object, allows overriding specific methods	When you want mostly real behavior with some overrides
Verify	Checks that a method was called with specific arguments	Verifying side effects (email sent, event published)

Mock vs Spy

```
MOCK (completely fake):                SPY (real object with overrides):
  List<String> mockList = mock();         List<String> realList = new ArrayList<>
();
  mockList.add("hello");                  List<String> spyList = spy(realList);
  mockList.size(); // Returns 0           spyList.add("hello");
  // add() was NOT actually called        spyList.size(); // Returns 1
                                          // add() WAS actually called
```

6.3 Mockito Annotations

Annotation	Purpose	Example
`@Mock`	Creates a mock object	`@Mock ProductRepository productRepo;`
`@InjectMocks`	Creates the class under test and injects mocks	`@InjectMocks OrderService orderService;`
`@Spy`	Creates a spy (partial mock)	`@Spy PricingService pricingService;`
`@Captor`	Captures arguments passed to mock methods	`@Captor ArgumentCaptor<Order> orderCaptor;`

`@ExtendWith(MockitoExtension.class)`	Enables Mockito annotations in JUnit 5	Class-level annotation

6.4 Code Implementation

```
@ExtendWith(MockitoExtension.class)  // Enable Mockito in JUnit 5
class OrderServiceTest {

    @Mock                               // Fake repository -- no real database
    private OrderRepository orderRepository;

    @Mock                               // Fake payment client -- no real API
calls
    private PaymentClient paymentClient;

    @Mock                               // Fake email service -- no real emails
    private EmailService emailService;

    @InjectMocks                        // Creates OrderService with all mocks
injected
    private OrderService orderService;

    @Captor
    private ArgumentCaptor<Order> orderCaptor;

    @Test
    @DisplayName("Should place order successfully")
    void testPlaceOrderSuccess() {
        // ARRANGE -- Configure mocks to return specific values
        Product product = new Product(1L, "Laptop", BigDecimal.valueOf(999.99),
10);

when(orderRepository.findProductById(1L)).thenReturn(Optional.of(product));
        when(paymentClient.charge(any(PaymentRequest.class)))
            .thenReturn(new PaymentResponse("SUCCESS", "txn-123"));
        when(orderRepository.save(any(Order.class)))
            .thenAnswer(invocation -> {
                Order order = invocation.getArgument(0);
                order.setId(100L);  // Simulate DB-generated ID
                return order;
            });

        // ACT -- Call the method under test
        OrderRequest request = new OrderRequest(1L, 2, "customer@test.com");
```

```java
        Order result = orderService.placeOrder(request);

        // ASSERT -- Verify the result
        assertNotNull(result);
        assertEquals(100L, result.getId());
        assertEquals("CONFIRMED", result.getStatus());
        assertEquals(BigDecimal.valueOf(1999.98), result.getTotalAmount());

        // VERIFY -- Ensure side effects happened
        verify(orderRepository).save(orderCaptor.capture());
        Order savedOrder = orderCaptor.getValue();
        assertEquals(2, savedOrder.getQuantity());

        verify(paymentClient).charge(any(PaymentRequest.class));
        verify(emailService).sendConfirmation("customer@test.com", 100L);
        verify(orderRepository, never()).delete(any());  // Should NOT delete
    }

    @Test
    @DisplayName("Should throw exception when product not found")
    void testPlaceOrderProductNotFound() {

when(orderRepository.findProductById(999L)).thenReturn(Optional.empty());

        assertThrows(ProductNotFoundException.class,
            () -> orderService.placeOrder(new OrderRequest(999L, 1,
"test@test.com")));

        // Verify payment was NEVER attempted
        verify(paymentClient, never()).charge(any());
        verify(emailService, never()).sendConfirmation(anyString(), anyLong());
    }
}
```

Code Explanation

Line	Purpose
`when(...).thenReturn(...)`	**Stubbing** -- tells the mock what to return when called
`when(...).thenAnswer(...)`	Dynamic stubbing -- return value based on input arguments
`any(Class.class)`	**Argument matcher** -- matches any argument of that type
`verify(mock).method()`	Verifies the mock method was called exactly once

`verify(mock, never()).method()`	Verifies the mock method was NEVER called
`verify(mock, times(2)).method()`	Verifies exact call count
`ArgumentCaptor.capture()`	Captures the actual argument for further assertions

Common Mistakes

Mistake	Problem	Fix
Mocking the class under test	You're testing the mock, not your code	Use `@InjectMocks` for the class being tested
Using `any()` mixed with exact values	Mockito throws error	Use matchers for ALL arguments or NONE
Not verifying side effects	Test passes but method doesn't send emails	Add `verify()` for important side effects
Over-mocking	Tests pass but the real integration fails	Use integration tests for critical paths

---

Part 7: Spring Boot Testing Architecture

7.1 How Spring Boot Supports Testing

Concept Explanation

Spring Boot provides a **layered testing strategy** with specialized annotations for each layer. Instead of loading the entire application context for every test, you load only what you need.

Why Specialized Test Annotations Exist

```
Loading FULL application context for every test:
  -> Starts ALL beans, ALL configurations, ALL connections
  -> Takes 10-30 seconds per test class
  -> 50 test classes x 20 seconds = 16+ minutes

Loading ONLY what you need:
  @WebMvcTest    -> Only controller + security beans (< 2 seconds)
  @DataJpaTest   -> Only JPA + embedded database (< 3 seconds)
  Unit test      -> No Spring context at all (milliseconds)
```

7.2 Test Annotation Reference

Annotation	What It Loads	Use Case	Speed
`@SpringBootTest`	Full application context	Integration tests, E2E	Slow (10-30s)
`@WebMvcTest`	Controllers + Security + MockMvc only	Controller layer tests	Fast (1-3s)
`@DataJpaTest`	JPA repositories + embedded DB	Repository layer tests	Fast (2-4s)
`@TestConfiguration`	Custom beans for testing only	Override production beans	N/A
`@MockBean`	Replace a Spring bean with a mock	Mock dependencies in Spring context	N/A

Architecture: Which Annotation Tests Which Layer

```
+-------------------------------------------------------------+
|                    @SpringBootTest                          |
|                 (Loads EVERYTHING)                          |
|                                                             |
|  +-----------------------------------------------------+  |
|  |              @WebMvcTest                            |  |
|  |         +--------------+                            |  |
|  |         |  Controller  |  <- Tests this layer       |  |
|  |         |  @RestController                           |  |
|  |         +------+-------+                            |  |
|  |                | @MockBean                          |  |
|  |         +------▼-------+                            |  |
|  |         |   Service    |  <- Mocked                 |  |
|  |         +--------------+                            |  |
|  +-----------------------------------------------------+  |
|                                                             |
|  +-----------------------------------------------------+  |
|  |              @DataJpaTest                           |  |
|  |         +--------------+                            |  |
|  |         |  Repository  |  <- Tests this layer       |  |
|  |         |  JpaRepository                             |  |
|  |         +------+-------+                            |  |
|  |                |                                    |  |
|  |         +------▼-------+                            |  |
|  |         |  H2 Embedded |  <- In-memory database     |  |
|  |         |  Database    |                            |  |
|  |         +--------------+                            |  |
|  +-----------------------------------------------------+  |
|                                                             |
|  +-----------------------------------------------------+  |
|  |              Unit Tests (No Spring)                 |  |
|  |         +--------------+                            |  |
|  |         |   Service    |  <- Tests this layer       |  |
|  |         +------+-------+                            |  |
|  |                | @Mock (Mockito)                    |  |
|  |         +------▼-------+                            |  |
|  |         | Mock Repo    |  <- Mocked (no Spring)     |  |
|  |         +--------------+                            |  |
|  +-----------------------------------------------------+  |
+-------------------------------------------------------------+
```

When to Use Each

Scenario	Best Annotation	Why

Test business logic in isolation	`@ExtendWith(MockitoExtension.class)`	No Spring needed, fastest
Test REST endpoint request/response	`@WebMvcTest`	Loads only web layer
Test JPA queries and database ops	`@DataJpaTest`	Loads only JPA with in-memory DB
Test full request flow end-to-end	`@SpringBootTest`	Loads everything

Part 8: Testing Controller Layer

8.1 Concept Explanation

What is Controller Layer Testing?

Testing that your REST controllers correctly handle HTTP requests, call the right service methods, and return proper HTTP responses (status codes, response bodies, headers).

Why Test Controllers Separately?

Controllers have specific responsibilities:

- Parse request parameters, path variables, request body
- Validate input (`@Valid`)
- Call service methods
- Return correct HTTP status codes (200, 201, 400, 404, 500)
- Serialize response to JSON

Testing Flow

```
Test sends HTTP request via MockMvc
         |
         ▼
+------------------+
|  DispatcherServlet|  Routes to correct controller
|  (Real)           |
+--------+---------+
         |
         ▼
+------------------+
|  Controller       |  Calls service method
|  (Real)           |
+--------+---------+
         |
         ▼
+------------------+
|  Service          |  Returns mocked response
|  (@MockBean)      |
+--------+---------+
         |
```

```
        ▼
MockMvc captures response -> Assert status, body, headers
```

8.2 Code Implementation

```java
@WebMvcTest(OrderController.class)  // Load ONLY OrderController
class OrderControllerTest {

    @Autowired
    private MockMvc mockMvc;  // Simulates HTTP requests without a real server

    @MockBean  // Replace real OrderService with a mock in Spring context
    private OrderService orderService;

    @Autowired
    private ObjectMapper objectMapper;  // JSON serializer

    @Test
    @DisplayName("POST /api/orders -- Should create order and return 201")
    void testCreateOrderSuccess() throws Exception {
        // ARRANGE
        OrderRequest request = new OrderRequest(1L, 2, "customer@test.com");
        Order mockOrder = new Order(100L, 1L, 2,
            BigDecimal.valueOf(199.98), "CONFIRMED");

        when(orderService.placeOrder(any(OrderRequest.class)))
            .thenReturn(mockOrder);

        // ACT & ASSERT
        mockMvc.perform(post("/api/orders")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isCreated())                    // HTTP 201
            .andExpect(jsonPath("$.id").value(100))
            .andExpect(jsonPath("$.status").value("CONFIRMED"))
            .andExpect(jsonPath("$.totalAmount").value(199.98));

        verify(orderService).placeOrder(any(OrderRequest.class));
    }

    @Test
    @DisplayName("GET /api/orders/{id} -- Should return 404 for non-existent
order")
    void testGetOrderNotFound() throws Exception {
```

```java
        when(orderService.findById(999L))
            .thenThrow(new OrderNotFoundException("Order not found: 999"));

        mockMvc.perform(get("/api/orders/999"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.error").value("Order not found: 999"));
    }

    @Test
    @DisplayName("POST /api/orders -- Should return 400 for invalid input")
    void testCreateOrderValidationError() throws Exception {
        // Missing required fields
        OrderRequest invalidRequest = new OrderRequest(null, -1, "");

        mockMvc.perform(post("/api/orders")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(invalidRequest)))
            .andExpect(status().isBadRequest());
    }

    @Test
    @DisplayName("GET /api/orders -- Should return paginated results")
    void testGetAllOrdersPaginated() throws Exception {
        Page<Order> mockPage = new PageImpl<>(List.of(
            new Order(1L, "CONFIRMED"), new Order(2L, "PENDING")),
            PageRequest.of(0, 20), 2);

        when(orderService.findAll(any(Pageable.class))).thenReturn(mockPage);

        mockMvc.perform(get("/api/orders")
                .param("page", "0")
                .param("size", "20"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.content.length()").value(2))
            .andExpect(jsonPath("$.totalElements").value(2));
    }
}
```

Best Practices

1. **Test all HTTP methods** -- GET, POST, PUT, DELETE for each endpoint
2. **Test all response codes** -- 200, 201, 400, 401, 403, 404, 500
3. **Test validation** -- send invalid input and assert 400
4. **Use** `@MockBean` -- keep controller tests isolated from service logic

---

Part 9: Testing Service Layer

9.1 Concept Explanation

What is Service Layer Testing?

Testing the **business logic** in your `@Service` classes. Service tests are pure unit tests using Mockito -- no Spring context, no database.

Why Service Tests are the Most Important

The service layer contains:

- Business rules and validation
- Orchestration of multiple repository calls
- Exception handling and error logic
- Transaction boundaries

If the service logic is wrong, the entire feature is wrong -- regardless of how well the controller and repository work.

Testing Flow

```
Test calls service method directly (no HTTP, no Spring)
         |
         ▼
+------------------+
|  Service         |  Contains business logic
|  (Real instance) |
+--------+---------+
         | calls
         ▼
+------------------+
|  Repository      |  Returns mock data
|  (@Mock)         |
+------------------+
```

9.2 Code Implementation

```java
@ExtendWith(MockitoExtension.class)  // Pure unit test -- no Spring
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @Mock
    private PasswordEncoder passwordEncoder;

    @Mock
    private EmailService emailService;

    @InjectMocks
    private UserService userService;

    @Test
    @DisplayName("Should register user successfully")
    void testRegisterSuccess() {
        // ARRANGE
        RegisterRequest request = new RegisterRequest(
            "Jane", "jane@test.com", "password123");

        when(userRepository.existsByEmail("jane@test.com")).thenReturn(false);

when(passwordEncoder.encode("password123")).thenReturn("hashed_password");
        when(userRepository.save(any(User.class))).thenAnswer(invocation -> {
            User user = invocation.getArgument(0);
            user.setId(1L);
            return user;
        });

        // ACT
        User result = userService.register(request);

        // ASSERT
        assertAll("registered user",
            () -> assertEquals(1L, result.getId()),
            () -> assertEquals("Jane", result.getName()),
            () -> assertEquals("jane@test.com", result.getEmail()),
            () -> assertEquals("hashed_password", result.getPassword()),
            () -> assertEquals(UserStatus.ACTIVE, result.getStatus())
        );

        // VERIFY side effects
        verify(emailService).sendWelcomeEmail("jane@test.com", "Jane");
        verify(userRepository).save(any(User.class));
```

```
    }

    @Test
    @DisplayName("Should throw exception for duplicate email")
    void testRegisterDuplicateEmail() {
        RegisterRequest request = new RegisterRequest(
            "Jane", "existing@test.com", "password123");

when(userRepository.existsByEmail("existing@test.com")).thenReturn(true);

        DuplicateEmailException exception = assertThrows(
            DuplicateEmailException.class,
            () -> userService.register(request));

        assertEquals("Email already registered: existing@test.com",
            exception.getMessage());

        // Verify save was NEVER called
        verify(userRepository, never()).save(any());
        verify(emailService, never()).sendWelcomeEmail(anyString(),
anyString());
    }

    @Test
    @DisplayName("Should update user profile")
    void testUpdateProfile() {
        User existingUser = new User(1L, "John", "john@test.com",
UserStatus.ACTIVE);

when(userRepository.findById(1L)).thenReturn(Optional.of(existingUser));
        when(userRepository.save(any(User.class))).thenAnswer(i ->
i.getArgument(0));

        UpdateProfileRequest request = new UpdateProfileRequest("Johnny", "555-
1234");
        User updated = userService.updateProfile(1L, request);

        assertEquals("Johnny", updated.getName());
        assertEquals("555-1234", updated.getPhone());
    }
}
```

Part 10: Testing Repository Layer

10.1 Concept Explanation

What is Repository Layer Testing?

Testing that your **JPA queries work correctly** -- both Spring Data derived queries and custom `@Query` methods. Repository tests use a **real database** (in-memory H2) to verify actual SQL execution.

Why Test the Repository Layer?

- **Derived query methods** (`findByStatusAndCreatedAtBefore`) may not generate the SQL you expect
- **Custom** `@Query` **JPQL** may have syntax errors that only appear at runtime
- **Native queries** may be database-specific and need validation
- **Entity mappings** (`@OneToMany` , `@ManyToMany`) may not cascade correctly

Testing Flow

```
@DataJpaTest starts:
  1. Scans for @Entity classes
  2. Creates H2 in-memory database
  3. Runs schema generation (DDL)
  4. Injects TestEntityManager
  5. Your test uses REAL repository methods
  6. After test: rolls back transaction (clean state)
```

10.2 Code Implementation

```java
@DataJpaTest  // Loads ONLY JPA beans + embedded H2 database
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
class OrderRepositoryTest {

    @Autowired
    private TestEntityManager entityManager;  // For inserting test data

    @Autowired
    private OrderRepository orderRepository;  // The repository under test
```

```
    @BeforeEach
    void setUp() {
        // Insert test data using TestEntityManager
        Customer customer = new Customer("John", "john@test.com");
        entityManager.persist(customer);

        Order order1 = new Order(customer, BigDecimal.valueOf(100),
"CONFIRMED");
        order1.setCreatedAt(LocalDateTime.now().minusDays(1));
        entityManager.persist(order1);

        Order order2 = new Order(customer, BigDecimal.valueOf(250), "PENDING");
        order2.setCreatedAt(LocalDateTime.now());
        entityManager.persist(order2);

        Order order3 = new Order(customer, BigDecimal.valueOf(50),
"CANCELLED");
        entityManager.persist(order3);

        entityManager.flush();
    }

    @Test
    @DisplayName("Should find orders by status")
    void testFindByStatus() {
        List<Order> confirmedOrders =
orderRepository.findByStatus("CONFIRMED");

        assertEquals(1, confirmedOrders.size());
        assertEquals(BigDecimal.valueOf(100),
confirmedOrders.get(0).getTotalAmount());
    }

    @Test
    @DisplayName("Should find orders by customer email")
    void testFindByCustomerEmail() {
        List<Order> orders =
orderRepository.findByCustomerEmail("john@test.com");

        assertEquals(3, orders.size());
    }

    @Test
    @DisplayName("Should calculate total revenue by status")
    void testCalculateTotalRevenueByStatus() {
        BigDecimal revenue =
orderRepository.calculateTotalRevenueByStatus("CONFIRMED");
```

```
        assertEquals(BigDecimal.valueOf(100), revenue);
    }

    @Test
    @DisplayName("Should find recent orders with pagination")
    void testFindRecentOrders() {
        Page<Order> page = orderRepository.findByStatusOrderByCreatedAtDesc(
            "PENDING", PageRequest.of(0, 10));

        assertEquals(1, page.getTotalElements());
        assertEquals("PENDING", page.getContent().get(0).getStatus());
    }

    @Test
    @DisplayName("Should return empty for non-existent status")
    void testFindByNonExistentStatus() {
        List<Order> orders = orderRepository.findByStatus("SHIPPED");

        assertTrue(orders.isEmpty());
    }
}
```

Best Practices

1. **Use** `TestEntityManager` to insert test data -- it participates in the same transaction
2. **Test custom queries** -- derived queries, @Query JPQL, and native queries
3. **Test edge cases** -- empty results, null parameters, pagination boundaries
4. **Each test gets a clean database** -- `@DataJpaTest` rolls back after each test

---

Part 11: Testcontainers

11.1 What are Testcontainers?

Concept Explanation

Testcontainers is a Java library that provides lightweight, disposable **real database containers** (Docker) for integration testing. Instead of using H2 (which behaves differently from MySQL/PostgreSQL), you test against the exact same database you use in production.

Why Testcontainers Over H2

H2 In-Memory Database	Testcontainers (Real DB)
H2 SQL dialect differs from MySQL/PostgreSQL	Runs the EXACT production database
Native queries may not work	Native queries work exactly as in production
JSON columns, stored procedures not supported	Full feature support
Tests pass locally, fail in production	Tests match production behavior

When to Use Testcontainers

- **Native SQL queries** that use database-specific syntax
- **Stored procedures** testing
- **Database migration** testing (Flyway/Liquibase)
- When H2 doesn't support your database features (JSONB, full-text search)

When NOT to Use

- Simple JPQL/derived queries -- H2 is sufficient and faster
- Unit tests -- don't need any database
- CI pipelines without Docker support

11.2 Architecture

```
Test Execution Flow with Testcontainers:

  JUnit starts test class
         |
```

```
        ▼
@Container annotation detected
        |
        ▼
Docker pulls MySQL/PostgreSQL image
        |
        ▼
Container starts (real database)
        |
        ▼
Spring connects to container (dynamic port)
        |
        ▼
Tests run against REAL database
        |
        ▼
Container destroyed after tests complete
```

11.3 Code Implementation

```java
@SpringBootTest
@Testcontainers  // Enable Testcontainers support
class OrderServiceIntegrationTest {

    @Container  // Start a real MySQL container
    static MySQLContainer<?> mysql = new MySQLContainer<>("mysql:8.0")
        .withDatabaseName("testdb")
        .withUsername("test")
        .withPassword("test");

    @DynamicPropertySource  // Override application properties with container
values
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", mysql::getJdbcUrl);
        registry.add("spring.datasource.username", mysql::getUsername);
        registry.add("spring.datasource.password", mysql::getPassword);
    }

    @Autowired
    private OrderService orderService;

    @Autowired
    private OrderRepository orderRepository;
```

```java
    @BeforeEach
    void setUp() {
        orderRepository.deleteAll();  // Clean state before each test
    }

    @Test
    @DisplayName("Should persist and retrieve order from real MySQL")
    void testFullOrderLifecycle() {
        // This test runs against a REAL MySQL database
        OrderRequest request = new OrderRequest(1L, 5, "customer@test.com");
        Order created = orderService.placeOrder(request);

        assertNotNull(created.getId());

        Order retrieved = orderService.findById(created.getId());
        assertEquals("CONFIRMED", retrieved.getStatus());
        assertEquals(5, retrieved.getQuantity());
    }
}
```

Maven Dependency

```xml
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>mysql</artifactId>
    <version>1.19.3</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.testcontainers</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>1.19.3</version>
    <scope>test</scope>
</dependency>
```

Part 12: Spring Security Testing

12.1 Concept Explanation

What is Security Testing?

Testing that your **secured endpoints** correctly enforce authentication and authorization rules. Verifying that unauthenticated users get 401, unauthorized users get 403, and authorized users can access their resources.

Why Security Testing is Critical

Without security tests:

- An endpoint might accidentally be public
- Role checks might be missing
- JWT validation might have gaps
- CORS configuration might allow unintended origins

Testing Flow

```
Security Test Flow:

  MockMvc sends request
       |
       +-- No credentials -> 401 Unauthorized ✓
       +-- Valid token + wrong role -> 403 Forbidden ✓
       +-- Valid token + correct role -> 200 OK ✓
```

12.2 Key Annotations

Annotation	Purpose	Example
`@WithMockUser`	Simulates an authenticated user with roles	`@WithMockUser(roles = "ADMIN")`
`@WithUserDetails`	Loads user from UserDetailsService	`@WithUserDetails("admin@test.com")`
`@WithAnonymousUser`	Simulates unauthenticated request	Tests that public endpoints work

12.3 Code Implementation

```
@WebMvcTest(AdminController.class)
class AdminControllerSecurityTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private AdminService adminService;

    // Test: Unauthenticated access should be rejected
    @Test
    @DisplayName("Should return 401 for unauthenticated request")
    void testUnauthenticatedAccess() throws Exception {
        mockMvc.perform(get("/api/admin/users"))
            .andExpect(status().isUnauthorized());
    }

    // Test: Authenticated but wrong role should be forbidden
    @Test
    @WithMockUser(roles = "USER")  // Simulate USER role
    @DisplayName("Should return 403 for non-admin user")
    void testForbiddenForNonAdmin() throws Exception {
        mockMvc.perform(get("/api/admin/users"))
            .andExpect(status().isForbidden());
    }

    // Test: Correct role should succeed
    @Test
    @WithMockUser(roles = "ADMIN")  // Simulate ADMIN role
    @DisplayName("Should return 200 for admin user")
    void testAdminAccess() throws Exception {
        when(adminService.getAllUsers()).thenReturn(List.of(
            new UserDTO(1L, "Admin User", "ADMIN")));

        mockMvc.perform(get("/api/admin/users"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.length()").value(1));
    }

    // Test: Method-level security
    @Test
    @WithMockUser(username = "user1@test.com", roles = "USER")
    @DisplayName("Should allow users to access their own profile only")
```

```
    void testOwnerOnlyAccess() throws Exception {
        mockMvc.perform(get("/api/users/profile"))
            .andExpect(status().isOk());
    }
}
```

Best Practices

1. **Test all access patterns** -- unauthenticated, wrong role, correct role
2. **Test method-level security** -- `@PreAuthorize` annotations
3. **Test custom security rules** -- ownership checks, IP restrictions
4. **Use `@WithMockUser`** for simple role tests, `@WithUserDetails` for complex user objects

---

Part 13: Code Coverage

13.1 What is Code Coverage?

Concept Explanation

Code coverage measures what percentage of your code is executed during tests. It answers: "How much of my codebase is verified by tests?"

Why Code Coverage Matters

Coverage Level	What It Means	Risk
0-30%	Most code is untested	Very high risk of bugs in production
30-60%	Core paths tested, edge cases missed	Medium risk
60-80%	Good coverage, most paths verified	Low risk -- recommended target
80-100%	Extensive coverage	Very low risk -- but diminishing returns above 90%

> ***Key Insight:*** *100% coverage does NOT mean bug-free code. Coverage measures lines executed, not correctness of assertions. You can have 100% coverage with zero assertions.*

13.2 JaCoCo -- Code Coverage Tool

Types of Coverage

Type	What It Measures	Why It Matters
Line Coverage	% of code lines executed	Basic measure
Branch Coverage	% of if/else branches taken	Catches untested conditions
Method Coverage	% of methods called	Identifies dead methods
Class Coverage	% of classes with any tests	Identifies untested classes

Maven Configuration

```
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.11</version>
    <executions>
        <execution>
            <goals>
                <goal>prepare-agent</goal>
            </goals>
        </execution>
        <execution>
            <id>report</id>
            <phase>test</phase>
            <goals>
                <goal>report</goal>
            </goals>
        </execution>
        <!-- Enforce minimum coverage -->
        <execution>
            <id>check</id>
            <goals>
                <goal>check</goal>
            </goals>
            <configuration>
                <rules>
                    <rule>
                        <element>BUNDLE</element>
                        <limits>
                            <limit>
                                <counter>LINE</counter>
                                <value>COVEREDRATIO</value>
                                <minimum>0.80</minimum>  <!-- 80% line coverage
-->
                            </limit>
                            <limit>
                                <counter>BRANCH</counter>
                                <value>COVEREDRATIO</value>
                                <minimum>0.70</minimum>  <!-- 70% branch
coverage -->
                            </limit>
                        </limits>
                    </rule>
                </rules>
            </configuration>
```

```
        </execution>
    </executions>
</plugin>
```

Running Coverage

```
# Generate coverage report
mvn clean test jacoco:report

# Report location: target/site/jacoco/index.html

# Fail build if coverage is below threshold
mvn clean verify   # jacoco:check runs during verify phase
```

Best Practices

1. **Aim for 70-80%** line coverage on service layer -- where business logic lives
2. **Focus on branch coverage** -- untested branches hide bugs
3. **Don't chase 100%** -- getters, setters, DTOs add coverage without value
4. **Exclude generated code** from coverage (Lombok, MapStruct)

Part 14: CI/CD Testing Integration

14.1 Why Tests in CI/CD?

Concept Explanation

Tests must run **automatically** on every commit and pull request. This prevents broken code from reaching the main branch and catches integration issues early.

Testing in the Build Pipeline

```
Developer pushes code
         |
         ▼
+--------------------------+
| CI Pipeline Triggered     |
|                           |
|  Stage 1: Compile         |  mvn compile
|  Stage 2: Unit Tests      |  mvn test (fast, no infra)
|  Stage 3: Integration     |  mvn verify (Testcontainers)
|  Stage 4: Coverage Check  |  JaCoCo minimum threshold
|  Stage 5: Build Artifact  |  mvn package (JAR/Docker)
|  Stage 6: Deploy          |  Only if all tests pass
|                           |
+--------------------------+
```

14.2 GitHub Actions Configuration

```yaml
name: Java CI with Tests

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
```

```yaml
    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: root
          MYSQL_DATABASE: testdb
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping -h 127.0.0.1"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=3

    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Cache Maven dependencies
        uses: actions/cache@v3
        with:
          path: ~/.m2
          key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}

      - name: Run Unit Tests
        run: mvn test -pl '!integration-tests'

      - name: Run Integration Tests
        run: mvn verify -pl integration-tests
        env:
          SPRING_DATASOURCE_URL: jdbc:mysql://localhost:3306/testdb

      - name: Upload Coverage Report
        uses: actions/upload-artifact@v3
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

14.3 Jenkins Pipeline

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Unit Tests') {
            steps { sh 'mvn test' }
            post {
                always { junit 'target/surefire-reports/*.xml' }
            }
        }
        stage('Integration Tests') {
            steps { sh 'mvn verify -P integration-tests' }
        }
        stage('Coverage') {
            steps { sh 'mvn jacoco:report' }
            post {
                always { jacoco(execPattern: 'target/jacoco.exec') }
            }
        }
        stage('Build') {
            steps { sh 'mvn package -DskipTests' }
        }
    }
}
```

14.4 Maven Build Lifecycle Integration

```
mvn test      -> Runs unit tests only (Surefire plugin)
mvn verify    -> Runs unit + integration tests (Failsafe plugin)
mvn package   -> Runs all tests, then packages JAR
mvn install   -> Runs all tests, packages, installs to local repo
```

Surefire vs Failsafe

Plugin	Purpose	File Pattern	Phase
Surefire	Unit tests	`*Test.java`, `*Tests.java`	`test`

Failsafe	Integration tests	`*IT.java`, `*IntegrationTest.java`	`verify`

Part 15: Production Testing Best Practices

15.1 Test Isolation

Concept

Each test must run independently -- no test should depend on the outcome of another test.

Rules

Rule	Why	How
No shared mutable state	Tests fail randomly when run in different order	Reset state in `@BeforeEach`
No database dependencies between tests	Test A inserts data that Test B reads	Each test inserts its own data
No external system dependencies	Tests fail when WiFi is off	Mock all external calls
No file system dependencies	Path issues across OS	Use temp directories

15.2 Test Readability

Naming Convention

```
// BAD: What does this test verify?
@Test void test1() { ... }
@Test void testOrder() { ... }

// GOOD: Method name describes scenario and expected outcome
@Test void shouldReturnConfirmedOrder_whenPaymentSucceeds() { ... }
@Test void shouldThrowException_whenProductOutOfStock() { ... }
@Test void shouldSendEmail_afterSuccessfulRegistration() { ... }
```

Arrange-Act-Assert Pattern

```java
@Test
void shouldCalculateDiscountForPremiumUser() {
    // ARRANGE -- Set up test data and dependencies
    User user = new User("Jane", Plan.PREMIUM);
    Product product = new Product("Laptop", BigDecimal.valueOf(1000));

    // ACT -- Execute the method under test
    BigDecimal discount = discountService.calculate(user, product);

    // ASSERT -- Verify the result
    assertEquals(BigDecimal.valueOf(100), discount); // 10% discount
}
```

15.3 Test Maintainability

Practice	Why
Use `@DisplayName` for human-readable names	Better test reports
Create test builders/factories for entities	Avoid duplicating object creation
Use constants for magic numbers	`assertEquals(MAX_RETRY_COUNT, result)` not `assertEquals(3, result)`
Keep tests small -- one assertion concept per test	Easier to debug failures

15.4 Test Performance

Strategy	Impact
Use `@WebMvcTest` instead of `@SpringBootTest` for controller tests	10x faster startup
Use `@ExtendWith(MockitoExtension.class)` instead of Spring for service tests	100x faster
Use `@Sql` to load test data instead of programmatic insert	Cleaner, reusable
Run slow integration tests in a separate profile	Fast feedback for unit tests

---

Part 16: Common Testing Mistakes

16.1 Testing Multiple Things in One Test

```
// BAD: Tests too many behaviors -- which one failed?
@Test
void testUserService() {
    User user = userService.create(request);
    assertNotNull(user);

    User found = userService.findById(user.getId());
    assertEquals("John", found.getName());

    userService.delete(user.getId());
    assertThrows(NotFoundException.class, () ->
userService.findById(user.getId()));
}

// GOOD: Each test verifies ONE behavior
@Test void shouldCreateUser() { ... }
@Test void shouldFindUserById() { ... }
@Test void shouldDeleteUser() { ... }
```

16.2 Overusing Mocks

```
// BAD: Mocking the repository AND the entity AND the DTO
when(repo.findById(1L)).thenReturn(Optional.of(mockUser));
when(mockUser.getName()).thenReturn("John");
when(mapper.toDTO(mockUser)).thenReturn(mockDTO);
when(mockDTO.getName()).thenReturn("John");
// You're testing mock behavior, not real behavior!

// GOOD: Use real objects, mock only external dependencies
User realUser = new User(1L, "John", "john@test.com");
when(repo.findById(1L)).thenReturn(Optional.of(realUser));
UserDTO result = userService.findById(1L);
assertEquals("John", result.getName());
```

16.3 Testing Implementation Instead of Behavior

```java
// BAD: Testing HOW the method works internally
@Test
void testFindUser() {
    userService.findById(1L);
    verify(repo).findById(1L);
    verify(mapper).toDTO(any());
    verify(cache).put(any(), any());
    // If you refactor internals, this test breaks even if behavior is correct
}

// GOOD: Testing WHAT the method returns
@Test
void testFindUser() {
    when(repo.findById(1L)).thenReturn(Optional.of(new User("John")));
    UserDTO result = userService.findById(1L);
    assertEquals("John", result.getName());
    // Doesn't care about internal implementation details
}
```

16.4 Not Testing Edge Cases

```
Common edge cases developers forget to test:

  ✗ null inputs
  ✗ empty strings
  ✗ empty collections
  ✗ zero values
  ✗ negative numbers
  ✗ boundary values (MAX_INT, MIN_INT)
  ✗ duplicate entries
  ✗ concurrent access
  ✗ timeout scenarios
  ✗ large datasets (pagination boundaries)
```

16.5 Ignoring Test Failures

```
// TERRIBLE: Catching and ignoring exceptions in tests
@Test
void testSomething() {
    try {
        riskyMethod();
    } catch (Exception e) {
        // Swallowed -- test always passes!
    }
}

// CORRECT: Let the exception fail the test
@Test
void testSomething() {
    assertDoesNotThrow(() -> riskyMethod());
}
```

Part 17: Complete Spring Boot Testing Project Structure

17.1 Recommended Folder Structure

```
src/
+-- main/
|   +-- java/com/company/orderservice/
|       +-- config/
|       |   +-- SecurityConfig.java
|       +-- controller/
|       |   +-- OrderController.java
|       |   +-- UserController.java
|       +-- dto/
|       |   +-- OrderRequest.java
|       |   +-- OrderResponse.java
|       +-- entity/
|       |   +-- Order.java
|       |   +-- User.java
|       +-- exception/
|       |   +-- GlobalExceptionHandler.java
|       |   +-- OrderNotFoundException.java
|       +-- repository/
|       |   +-- OrderRepository.java
|       |   +-- UserRepository.java
|       +-- service/
|           +-- OrderService.java
|           +-- UserService.java
|
+-- test/
|   +-- java/com/company/orderservice/
|       +-- controller/                    <- Controller tests (@WebMvcTest)
|       |   +-- OrderControllerTest.java
|       |   +-- UserControllerTest.java
|       +-- service/                       <- Service tests (Mockito only)
|       |   +-- OrderServiceTest.java
|       |   +-- UserServiceTest.java
|       +-- repository/                    <- Repository tests (@DataJpaTest)
|       |   +-- OrderRepositoryTest.java
|       |   +-- UserRepositoryTest.java
|       +-- integration/                   <- Full integration tests
|       |   +-- OrderIntegrationTest.java
```

```
|       |   +-- BaseIntegrationTest.java   <- Shared Testcontainers config
|       +-- security/                      <- Security tests
|       |   +-- SecurityConfigTest.java
|       +-- util/                          <- Test utilities
|           +-- TestDataBuilder.java       <- Factory for test objects
|           +-- TestConstants.java         <- Shared test constants
|
|   +-- resources/
|       +-- application-test.yml           <- Test-specific configuration
|       +-- data/
|           +-- test-orders.sql            <- SQL test data scripts
```

17.2 Purpose of Each Test Directory

Directory	Annotation	Tests	Dependencies
`controller/`	`@WebMvcTest`	HTTP request/response, validation, status codes	Service mocked via `@MockBean`
`service/`	`@ExtendWith(MockitoExtension.class)`	Business logic, validation, exception handling	Repository mocked via `@Mock`
`repository/`	`@DataJpaTest`	JPA queries, entity mappings, cascading	Embedded H2 database
`integration/`	`@SpringBootTest`	Full flow: controller -> service -> repository	Testcontainers or embedded DB
`security/`	`@WebMvcTest` + `@WithMockUser`	Auth/authz rules, role-based access	Security filter chain

17.3 Test Data Builder Pattern

```java
// Avoid repeating object creation in every test
public class TestDataBuilder {

    public static User.UserBuilder aUser() {
        return User.builder()
            .id(1L)
            .name("John Doe")
            .email("john@test.com")
            .status(UserStatus.ACTIVE)
            .createdAt(LocalDateTime.now());
```

```java
    }

    public static OrderRequest.OrderRequestBuilder anOrderRequest() {
        return OrderRequest.builder()
            .productId(1L)
            .quantity(2)
            .customerEmail("customer@test.com");
    }

    public static Order.OrderBuilder anOrder() {
        return Order.builder()
            .id(100L)
            .status("CONFIRMED")
            .totalAmount(BigDecimal.valueOf(199.98))
            .createdAt(LocalDateTime.now());
    }
}

// Usage in tests:
User user = TestDataBuilder.aUser().name("Jane").build();
Order order = TestDataBuilder.anOrder().status("PENDING").build();
```

Part 18: Interview Questions & Answers (100+)

JUnit Fundamentals (Q1-Q20)

Q1. What is the difference between JUnit 4 and JUnit 5? JUnit 5 has a modular architecture (Platform + Jupiter + Vintage), supports multiple extensions via `@ExtendWith`, supports lambda-based assertions, has richer parameterized testing, and requires Java 8+. JUnit 4 is monolithic and uses `@RunWith` for a single runner.

Q2. What is the JUnit Platform? The foundation layer that provides the Launcher API for discovering and executing tests. It allows different test engines (Jupiter, Vintage, third-party) to run on the same platform.

Q3. What is the difference between `@BeforeAll` and `@BeforeEach`? `@BeforeAll` runs once before all tests in a class (must be static), used for expensive one-time setup like database connections. `@BeforeEach` runs before every test method, used to reset state for test isolation.

Q4. Why must `@BeforeAll` be static? Because it runs before any test instance is created. JUnit 5 creates a new instance for each test method by default (`PER_METHOD` lifecycle).

Q5. What is `assertAll()` and when should you use it? `assertAll()` groups multiple assertions together and executes ALL of them, reporting all failures at once. Use it when verifying multiple fields of a DTO/entity so you see all failures, not just the first one.

Q6. How does `assertThrows()` work? It executes the provided executable and asserts that the specified exception type is thrown. It returns the exception, allowing further assertions on the exception message or properties.

Q7. What is `@DisplayName` used for? It provides a human-readable test name that appears in test reports instead of the method name. Useful for describing the business scenario being tested.

Q8. What is `@Nested` used for? It allows grouping related tests within a test class, creating a logical hierarchy. For example, grouping all "delete" tests under a nested class within `UserServiceTest`.

Q9. What is the test lifecycle in JUnit 5? `@BeforeAll` -> (`@BeforeEach` -> `@Test` -> `@AfterEach`) x N tests -> `@AfterAll`.

Q10. What is `@TestMethodOrder`? It controls the execution order of test methods. Options include `OrderAnnotation` (using `@Order`), `MethodName` (alphabetical), and `Random`.

Q11. What is `@Disabled` used for? Temporarily skips a test method or class. Should always include a reason: `@Disabled("Waiting for bug fix #123")`.

Q12. What is parameterized testing? Running the same test logic with different input data. Uses `@ParameterizedTest` with sources like `@ValueSource`, `@CsvSource`, `@MethodSource`.

Q13. What is `@CsvSource` vs `@CsvFileSource`? `@CsvSource` provides inline CSV data in the annotation. `@CsvFileSource` loads data from an external CSV file, useful for large datasets.

Q14. When should you use `@MethodSource`? When test parameters are complex objects that can't be represented as CSV strings. The factory method returns a `Stream<Arguments>`.

Q15. What is `@RepeatedTest`? Runs a test method multiple times. Useful for testing randomized behavior or verifying consistency.

Q16. What is `@Timeout` in JUnit 5? Fails the test if it doesn't complete within the specified duration. Useful for detecting performance regressions.

Q17. What is `@Tag` used for? Categorizes tests (e.g., "slow", "integration", "smoke"). Allows running specific test categories: `mvn test -Dgroups=smoke`.

Q18. What is `@TempDir`? JUnit 5 extension that creates a temporary directory for the test and cleans it up after. Useful for file I/O tests.

Q19. How do you test private methods? You don't. Test them through public methods. If a private method is complex enough to need its own test, it should be extracted to a separate class.

Q20. What is the Arrange-Act-Assert pattern? A test structure: Arrange (set up data), Act (call the method), Assert (verify the result). Makes tests readable and consistent.

Mockito (Q21-Q45)

Q21. What is the difference between `@Mock` and `@InjectMocks`? `@Mock` creates a mock object. `@InjectMocks` creates a real instance of the class and injects all `@Mock` objects into it via constructor or setter injection.

Q22. What is the difference between `mock()` and `spy()`? A `mock` is completely fake -- all methods return defaults. A `spy` wraps a real object -- methods call real implementations unless explicitly stubbed.

Q23. What does `when().thenReturn()` do? Stubs a mock method to return a specific value when called. For example: `when(repo.findById(1L)).thenReturn(Optional.of(user))`.

Q24. What is `when().thenAnswer()`? Dynamic stubbing where the return value is computed based on the method arguments. Useful for simulating ID generation.

Q25. What is `verify()` used for? Verifies that a mock method was called with specific arguments. Used for testing side effects like sending emails or publishing events.

Q26. What is `verify(mock, never())`? Asserts that a method was NOT called. Used to verify that error paths don't trigger unwanted side effects.

Q27. What is `verify(mock, times(n))`? Asserts exact call count. `times(0)` is equivalent to `never()`.

Q28. What is `ArgumentCaptor`? Captures the actual argument passed to a mock method for further assertions. Useful for verifying the content of saved entities.

Q29. What are argument matchers? `any()`, `anyString()`, `anyLong()`, `eq()` etc. Match arguments flexibly instead of exact values. Rule: if one argument uses a matcher, ALL arguments must use matchers.

Q30. What is `@ExtendWith(MockitoExtension.class)`? Enables Mockito annotations (`@Mock`, `@InjectMocks`, etc.) in JUnit 5. Replaces JUnit 4's `@RunWith(MockitoJUnitRunner.class)`.

Q31. When should you NOT use mocking? Don't mock value objects, DTOs, or simple data classes. Don't mock the class under test. Don't mock third-party libraries you don't control -- use integration tests.

Q32. What is `doReturn().when()` vs `when().thenReturn()`? `doReturn().when()` doesn't actually invoke the method (safe for spies). `when().thenReturn()` invokes the method first, which can cause issues with spies.

Q33. What is `@Captor`? Annotation equivalent of `ArgumentCaptor.forClass(MyClass.class)`. Cleaner way to declare captors.

Q34. How do you mock void methods? Use `doNothing().when(mock).voidMethod()` or `doThrow(exception).when(mock).voidMethod()`.

Q35. What is `InOrder` verification? Verifies that mock methods were called in a specific order: `InOrder inOrder = inOrder(mock1, mock2);`.

Q36. What is `BDDMockito`? BDD-style Mockito API: `given().willReturn()` instead of `when().thenReturn()`. `then().should()` instead of `verify()`.

Q37. How do you mock static methods? Use `mockStatic(ClassName.class)` with try-with-resources in Mockito 3.4+. Avoid if possible -- static methods indicate design issues.

Q38. What is `@MockBean` vs `@Mock`? `@Mock` is pure Mockito (no Spring). `@MockBean` replaces a Spring bean in the application context with a mock. Use `@MockBean` in Spring tests.

Q39. Can you mock final classes? Yes, with Mockito 2+ by adding `mockito-extensions/org.mockito.plugins.MockMaker` file with `mock-maker-inline`.

Q40. What is `lenient()` stubbing? By default, Mockito fails if a stubbing isn't used. `lenient().when(...)` suppresses this error. Use sparingly.

Q41. How do you reset mocks? `Mockito.reset(mock)` or use `@BeforeEach` with fresh mock creation. Resetting is usually a code smell.

Q42. What is `verifyNoMoreInteractions()`? Asserts that no other methods were called on the mock beyond what was already verified. Use carefully -- can make tests brittle.

Q43. How do you mock a builder pattern? Use `RETURNS_SELF` answer: `mock(Builder.class, RETURNS_SELF)`.

Q44. What is `@Spy` annotation? Creates a spy (partial mock) of a real object. The real methods are called unless explicitly stubbed.

Q45. How do you verify timeout interactions? `verify(mock, timeout(1000)).method()` -- waits up to 1 second for the interaction to occur. Useful for async testing.

Spring Boot Testing (Q46-Q75)

Q46. What does `@SpringBootTest` do internally? Loads the full Spring ApplicationContext, auto-configures embedded servers, registers all beans, and sets up the complete application environment.

Q47. What is `@WebMvcTest` vs `@SpringBootTest`? `@WebMvcTest` loads ONLY the web layer (controllers, filters, security). `@SpringBootTest` loads EVERYTHING. Use `@WebMvcTest`

for faster controller tests.

Q48. What is MockMvc? A test utility that simulates HTTP requests without starting a real server. It tests the full Spring MVC pipeline (serialization, validation, exception handling).

Q49. What is `@DataJpaTest`? Loads only JPA components -- entities, repositories, EntityManager, and an embedded database. Uses `@Transactional` to roll back after each test.

Q50. What is `TestEntityManager`? A Spring-provided alternative to EntityManager for tests. Provides convenience methods like `persistAndFlush()` and `find()`.

Q51. What is `@AutoConfigureTestDatabase`? Controls whether to replace the configured database with an embedded one. `Replace.NONE` keeps the original datasource.

Q52. What is `@TestConfiguration`? Defines additional beans or overrides for testing only. Unlike `@Configuration`, it's not picked up by component scanning automatically.

Q53. What is `@DynamicPropertySource`? Dynamically adds properties to the Spring Environment at test startup. Used with Testcontainers to set database URLs from the running container.

Q54. How to test async methods? Use `@SpringBootTest` with `Awaitility` library: `await().atMost(5, SECONDS).until(() -> result.isDone())`.

Q55. What is `@ActiveProfiles`? Activates specific Spring profiles for the test: `@ActiveProfiles("test")` loads `application-test.yml`.

Q56. What is `@Sql` annotation? Executes SQL scripts before/after a test. `@Sql("/test-data.sql")` loads test data from a file.

Q57. How to test `@Scheduled` methods? Don't test the scheduling -- test the business method directly. Use `@SpringBootTest` with `awaitility` to verify the scheduled execution if needed.

Q58. What is `@WebFluxTest`? Similar to `@WebMvcTest` but for reactive WebFlux controllers. Uses `WebTestClient` instead of `MockMvc`.

Q59. How to test error handling with `@ControllerAdvice`? Use `@WebMvcTest` and trigger exceptions from `@MockBean`. Assert the error response structure and status code.

Q60. What is `TestRestTemplate`? Used in `@SpringBootTest(webEnvironment = RANDOM_PORT)` to make real HTTP requests to the running application.

Q61. What is `@JsonTest`? Tests JSON serialization/deserialization only. Loads `ObjectMapper`, `JacksonTester`, no web layer.

Q62. How do you test file upload endpoints? Use `MockMultipartFile` with `MockMvc.perform(multipart("/upload").file(mockFile))`.

Q63. How to test request validation (`@Valid`)? Send invalid JSON via MockMvc and assert `status().isBadRequest()` with specific validation error messages.

Q64. What is `@Import` in test context? Imports specific configuration classes into the test context: `@Import(SecurityConfig.class)`.

Q65. How to test pagination? Send page/size params via MockMvc and assert `$.content.length()`, `$.totalElements`, `$.totalPages`.

Q66. What is `@WithMockUser`? Simulates an authenticated user with specified username, roles, and authorities for the duration of the test.

Q67. What is `@WithUserDetails`? Loads a user from `UserDetailsService` by username. Uses the actual user object, including custom authorities.

Q68. How to test CORS configuration? Send a preflight OPTIONS request with `Origin` header and assert `Access-Control-Allow-Origin` in response.

Q69. How to test rate limiting? Send multiple requests in a loop and assert that requests beyond the limit return 429.

Q70. What is `@RestClientTest`? Tests REST client classes (RestTemplate/WebClient). Auto-configures `MockRestServiceServer` to mock external API responses.

Q71. How to test event publishing? Use `@SpyBean` on `ApplicationEventPublisher` and verify event was published, or use `ApplicationEvents` JUnit extension.

Q72. How to test caching? Call a method twice, verify the repository was called only once (second call served from cache).

Q73. How to test `@Transactional` rollback? Create a test that triggers an exception mid-transaction and verify no data was persisted.

Q74. What is `@Commit`? Used in test classes to commit the transaction instead of rolling back. Use for verifying data persistence.

Q75. How to test multi-tenant applications? Set tenant context in `@BeforeEach`, verify data isolation by checking that Tenant A cannot see Tenant B's data.

Integration Testing & Code Coverage (Q76-Q100)

Q76. What are Testcontainers? A Java library that provides lightweight Docker containers for integration testing. Runs real databases (MySQL, PostgreSQL) during tests.

Q77. What is `@Container`? Marks a field as a Testcontainers container that should be started/stopped with the test lifecycle.

Q78. What is `@Testcontainers`? Enables Testcontainers support -- manages container lifecycle, starts before tests, stops after.

Q79. When should you use H2 vs Testcontainers? Use H2 for faster tests with standard JPQL. Use Testcontainers when using native SQL, stored procedures, or database-specific features.

Q80. What is JaCoCo? A Java code coverage library that measures line, branch, method, and class coverage during test execution.

Q81. What is the difference between line coverage and branch coverage? Line coverage measures % of lines executed. Branch coverage measures % of decision branches (if/else) taken. Branch coverage is more important for finding bugs.

Q82. What is a good code coverage target? 70-80% for service layer, 50-60% for controller layer, 80%+ for critical business logic. Don't chase 100% -- diminishing returns.

Q83. What is the testing pyramid? A strategy suggesting: many unit tests (70-80%), some integration tests (15-20%), few E2E tests (5-10%).

Q84. What is the difference between Surefire and Failsafe plugins? Surefire runs unit tests (`*Test.java`) during `test` phase. Failsafe runs integration tests (`*IT.java`) during `verify` phase.

Q85. What is test isolation? Each test runs independently without depending on other tests' state or execution order.

Q86. What is mutation testing? A technique that introduces bugs (mutations) into code and checks if tests catch them. If tests pass with the mutation, they're insufficient. Tools: PIT.

Q87. What is contract testing? Testing API contracts between services. Ensures a producer's API matches what consumers expect. Tools: Pact, Spring Cloud Contract.

Q88. What is the difference between smoke testing and regression testing? Smoke testing verifies basic functionality works (quick check). Regression testing verifies that new changes didn't break existing functionality.

Q89. What is test flakiness? Tests that sometimes pass and sometimes fail without code changes. Common causes: timing issues, shared state, external dependencies.

Q90. How to handle test data in integration tests? Use `@Sql` scripts, TestEntityManager, test builders, or database migrations. Clean up with `@AfterEach` or `@Transactional` rollback.

Q91. What is BDD (Behavior-Driven Development)? Testing approach using Given-When-Then format. Tools: Cucumber, BDDMockito.

Q92. What is TDD (Test-Driven Development)? Write tests FIRST, then implement code to make tests pass, then refactor. Red -> Green -> Refactor cycle.

Q93. What is snapshot testing? Comparing current output against a stored "snapshot" of expected output. Common in frontend testing, less common in Java.

Q94. How do you test microservice communication? Use `@RestClientTest` with `MockRestServiceServer` for RestTemplate. Use WireMock for more complex scenarios.

Q95. What is WireMock? A library for mocking HTTP APIs. Runs a lightweight HTTP server that returns configured responses for specific requests.

Q96. How should you organize test packages? Mirror the main source structure. `com.company.service.OrderService` -> `com.company.service.OrderServiceTest`.

Q97. What is `@SpyBean`? Creates a spy of a Spring bean -- calls real methods but allows verification and selective stubbing.

Q98. How do you test circuit breakers? Use `@SpringBootTest`, trigger failures in the mocked dependency, and verify the fallback method is called.

Q99. What are the FIRST principles of good tests? Fast, Independent, Repeatable, Self-validating, Timely. Every test should embody these principles.

Q100. What is the biggest testing mistake in production systems? Testing implementation details instead of behavior. Tests should verify WHAT the code does, not HOW it does it. This makes tests resilient to refactoring.

Q101. When should you use `@SpringBootTest` over `@WebMvcTest`? Only when you need the full application context -- e.g., testing a flow that involves controller -> service -> repository ->

database. For pure controller tests, `@WebMvcTest` is faster and sufficient.

Q102. How do you test Kafka consumers/producers? Use `@EmbeddedKafka` annotation or Testcontainers Kafka module to run a real Kafka broker during tests.

Q103. What is `ApplicationContextRunner`? A Spring Boot test utility for testing auto-configuration. Simulates different configuration scenarios without starting a full context.

Q104. How do you handle slow tests in CI/CD? Separate unit tests (fast, run always) from integration tests (slow, run on merge). Use Maven profiles or `@Tag` filtering.

Q105. What is the role of `@Transactional` in tests? It wraps each test in a transaction and rolls back after completion, ensuring a clean database state for the next test.

Part 19: Hands-on Practice Exercises

Exercise 1: Test a REST API Controller

Scenario: You have a `ProductController` with these endpoints:

- `GET /api/products` -> returns all products
- `GET /api/products/{id}` -> returns one product or 404
- `POST /api/products` -> creates a product (validates name, price)
- `PUT /api/products/{id}` -> updates a product
- `DELETE /api/products/{id}` -> deletes a product

Task: Write `@WebMvcTest` tests covering:

1. Successful GET all products (200 with JSON array)
2. GET by ID -- found (200) and not found (404)
3. POST with valid data (201) and invalid data (400)
4. DELETE existing product (204)

Hint: Use `@MockBean` for `ProductService`, MockMvc for requests.

Exercise 2: Test Business Logic with Mockito

Scenario: `DiscountService` calculates discounts based on rules:

- Regular customers: no discount
- Premium customers: 10% discount
- Enterprise customers: 20% discount
- Orders over $1000: additional 5% discount
- Black Friday: double all discounts

Task: Write unit tests with Mockito covering:

1. All customer types with correct discount
2. Large order additional discount
3. Combination of customer discount + large order discount
4. Black Friday scenario
5. Edge case: zero amount order

Hint: Mock `CustomerRepository` and `ConfigService` (for Black Friday flag).

Exercise 3: Test Repository with @DataJpaTest

Scenario: `OrderRepository` has these custom queries:

```
List<Order> findByStatusAndCreatedAtAfter(String status, LocalDateTime date);

@Query("SELECT SUM(o.amount) FROM Order o WHERE o.customerId = :id")
BigDecimal calculateTotalSpent(@Param("id") Long customerId);

@Query(value = "SELECT * FROM orders WHERE JSON_CONTAINS(tags, :tag)",
nativeQuery = true)
List<Order> findByTag(@Param("tag") String tag);
```

Task: Write `@DataJpaTest` tests covering:

1. Find orders by status and date range
2. Calculate total spent -- with orders and without orders
3. Verify the aggregation query returns correct sum
4. Edge case: customer with no orders returns null or zero

Exercise 4: Integration Test with Testcontainers

Scenario: Full order placement flow -- user places an order, payment is processed, inventory is updated.

Task: Write a `@SpringBootTest` integration test:

1. Start a MySQL container with `@Testcontainers`
2. Insert test data (customer, product with stock)
3. Call `OrderService.placeOrder()` with valid request
4. Verify order is persisted in database
5. Verify inventory was decreased
6. Verify `OrderCreatedEvent` was published

Exercise 5: Security Testing

Scenario: Application has these access rules:

- `/api/public/**` -> accessible by everyone
- `/api/users/profile` -> accessible by authenticated users
- `/api/admin/**` -> accessible only by ADMIN role
- `/api/orders` POST -> requires USER role
- `/api/orders` DELETE -> requires ADMIN role

Task: Write security tests covering:

1. Public endpoint accessible without authentication
2. Protected endpoint returns 401 without token
3. User role can POST orders but cannot DELETE
4. Admin role can DELETE orders
5. Invalid token returns 401

Exercise 6: Build a Complete Test Suite

Put it all together. For a `UserService` that handles registration:

1. **Service test (Mockito):** Test `register()` method -- happy path, duplicate email, invalid input
2. **Controller test (MockMvc):** Test `POST /api/users/register` -- 201, 400, 409 (conflict)
3. **Repository test (@DataJpaTest):** Test `existsByEmail()`, `findByEmail()`
4. **Integration test:** Full registration flow -- request -> controller -> service -> database
5. **Security test:** Registration endpoint is public, profile endpoint requires auth

Success criteria: All tests pass, coverage > 80% on `UserService`.

End of Guide

> *This guide covered testing from fundamentals to production best practices. The key takeaway: **test behavior, not implementation**. Write tests that verify what your code does, not how it does it. This creates a test suite that gives you confidence to refactor and evolve your codebase.*